PROJETO NEXORA · MÓDULO 1

Estação Meteorológica

ESP32 + Wi-Fi + dados reais de Alagoinhas-BA
TFT 2,8" colorida · OLED 128x128 · DHT22 · BMP180 · LED RGB





Material de estudo: analogias, teoria, cálculos explicados, montagem, código e lista de compras.
Simulação no Wokwi · Outubro de 2026

O que vamos construir

Este é o primeiro módulo de "sentir o ambiente" do Nexora. A estação junta duas fontes de informação: **dados reais da internet** sobre Alagoinhas (previsão, sol, UV, qualidade do ar) e **medições locais** feitas por sensores na sua casa. Tudo aparece num painel de 6 páginas numa tela colorida, com um resumo permanente numa OLED e um LED que muda de cor quando há algo para prestar atenção.

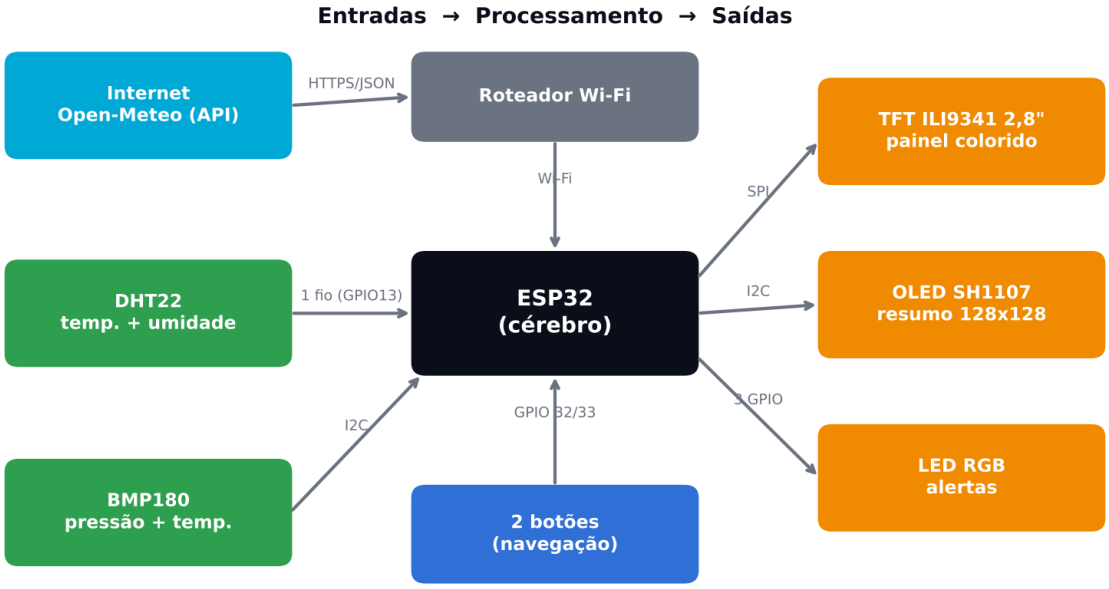


Figura 1 – Diagrama de blocos: o ESP32 recebe informação (esquerda), processa e mostra (direita).

Roteiro deste material

Parte	Assunto	Você vai aprender
1	Internet das Coisas	Wi-Fi, API, HTTP, HTTPS, JSON e NTP com analogias
2	Meteorologia	Temperatura, umidade, ponto de orvalho, pressão, vento, UV, qualidade do ar (com cálculos)
3	Eletrônica	ESP32, barramentos SPI e I2C, sensores, telas, pull-up, LED RGB e resistores
4	Montagem no Wokwi	Ligações pino a pino e como rodar a simulação
5	O código explicado	Estrutura, loop sem travar (millis), alertas, desenho na tela
6	As telas	O que cada página mostra e como ler
7	Do Wokwi para o mundo real	Lista de compras, diferenças e cuidados
8	Problemas e próximos passos	Solução de problemas e evoluções para o Nexora

Como estudar este PDF (Técnica Feynman)

Cada assunto começa com uma **analogia simples**, depois vem a **explicação técnica** e, quando há conta, um **cálculo passo a passo** com o porquê de cada passo. Ao terminar uma parte, feche o PDF e tente explicar com suas palavras no caderno. Onde travar, é exatamente ali que você volta para reler.

Parte 1 · Como o ESP32 conversa com a internet

1.1 Wi-Fi: a porta de entrada

> **Analogia do dia a dia: o carteiro do bairro**
>
> Pense no roteador como a agência dos Correios do seu bairro. Cada casa (aparelho) recebe um endereço (o **IP**, por exemplo 192.168.0.15). O ESP32 entrega a carta (o pedido de dados) para a agência, e ela se encarrega de levar até a cidade certa (o servidor da Open-Meteo) e trazer a resposta de volta.

O ESP32 tem um rádio de 2,4 GHz embutido. No modo **estação** (`WIFI_STA`) ele se conecta a um roteador como um celular faria. No Wokwi, existe uma rede virtual chamada `Wokwi-GUEST`, sem senha, que dá acesso de verdade à internet. O canal 6 é informado só para conectar mais rápido (o ESP32 não precisa procurar a rede em todos os canais).

```
WiFi.mode(WIFI_STA);                          // modo "cliente", como um celular
WiFi.begin("Wokwi-GUEST", "", 6);             // rede, senha, canal
while (WiFi.status() != WL_CONNECTED) delay(250);
```

1.2 API e HTTP: pedir dados como quem pede comida

> **Analogia do dia a dia: o garçom do restaurante**
>
> Você (ESP32) não entra na cozinha (o servidor) para pegar a comida. Você olha o **cardápio** (a documentação da API), faz o **pedido** ao garçom (a requisição HTTP) e recebe o **prato** (a resposta). Se o pedido estiver errado, o garçom volta dizendo "não temos isso" (um código de erro).

API (Interface de Programação de Aplicações) é um "cardápio" de dados que um serviço oferece para outros programas. A **Open-Meteo** é gratuita para uso não comercial e não exige cadastro nem chave, por isso foi a escolhida. O pedido é feito com o método **GET** do protocolo HTTP, e tudo o que queremos vai dentro da própria URL, como parâmetros depois do ?, separados por &:

Pedaço da URL	Significado
https://api.open-meteo.com/v1/forecast	endereço do "balcão" de previsão do tempo
latitude=-12.1356 & longitude=-38.4192	onde fica Alagoinhas (sul do equador e oeste de Greenwich, por isso negativos)
current=temperature_2m, relative_humidity_2m, ...	condições de agora (calculadas a cada 15 min pelo modelo)
hourly=temperature_2m, precipitation_probability, weather_code	dados hora a hora
forecast_hours=12	só as próximas 12 horas, a partir da hora atual (economiza memória)

Pedaço da URL	Significado
daily=temperature_2m_max, sunrise, sunset, uv_index_max, ...	resumo de cada dia
forecast_days=5	cinco dias
timezone=America%2FBahia	horários já no fuso da Bahia (%2F é a barra "/" codificada para URL)

O servidor responde com um **código de status**: **200** = deu certo, **400** = pedido mal escrito (por exemplo, nome de variável errado), **404** = endereço não existe, **5xx** = problema no servidor. O código só continua se recebeu 200.

1.3 HTTPS: a carta lacrada

Analogia do dia a dia: envelope com lacre

HTTP é um cartão-postal: qualquer um no caminho pode ler. HTTPS é uma carta num envelope lacrado (criptografia TLS). Além disso, o envelope vem com um "selo de autenticidade" (o certificado) que prova que quem respondeu é mesmo a Open-Meteo.

No código usamos `client.setInsecure()`. Isso significa: "*use o envelope lacrado, mas não confira o selo*". Os dados continuam criptografados, mas o ESP32 não verifica se o servidor é legítimo. Para dados públicos de clima o risco é baixo e simplifica muito (certificados expiram e precisariam ser atualizados no código).

Quando NÃO usar setInsecure()

Em projetos do Nexora que enviem senhas, abram portas, liguem cargas ou mexam com dados pessoais. Nesses casos, grave o certificado raiz (CA) do servidor com `client.setCACert(...)`.

1.4 JSON: a ficha preenchida

Analogia do dia a dia: a ficha de cadastro

Uma ficha tem **campos** com nome e valor: "Nome: Maria", "Cidade: Alagoinhas". JSON é uma ficha escrita de um jeito que computador entende: `"temperature_2m": 30.8`. Chaves `{ }` agrupam campos (uma ficha dentro da outra) e colchetes `[ ]` fazem listas.

Um trecho da resposta real tem este formato (valores de exemplo):

```
{
  "current": { "time": "2026-10-11T14:30", "temperature_2m": 30.8,
               "relative_humidity_2m": 62, "weather_code": 2, "uv_index": 7.2 },
  "hourly":  { "time": ["2026-10-11T14:00", "2026-10-11T15:00", ...],
               "temperature_2m": [30.8, 31.2, ...],
               "precipitation_probability": [10, 15, ...] },
  "daily":   { "time": ["2026-10-11", ...], "sunrise": ["2026-10-11T05:21", ...] }
}
```

A biblioteca **ArduinoJson** transforma esse texto numa estrutura navegável. O operador | define um **valor padrão** caso o campo não exista ou venha vazio (null), o que acontece de vez em quando com a chance de chuva:

```
wx.temp    = doc["current"]["temperature_2m"] | NAN;   // se faltar: "não é um número"
wx.hProb[i] = h["precipitation_probability"][i] | 0;  // se vier null: 0 %
```

Repare na hora "2026-10-11T14:00": a posição 11 do texto é onde começa "14". Por isso o código faz atoi(t + 11): pula 11 caracteres e converte "14:00" no número 14 (atoi para de ler quando encontra o ":").

1.5 NTP: acertar o relógio pela internet

Analogia do dia a dia: o relógio da rodoviária

O relógio do ESP32 começa zerado toda vez que liga, como um relógio sem pilha. O NTP é como perguntar a hora certa para o relógio oficial da rodoviária. O ESP32 pergunta ao servidor pool.ntp.org, recebe a hora mundial (UTC) e aplica o fuso: −3 horas.

Cálculo passo a passo: o fuso horário em segundos

1) A função configTime quer o fuso em **segundos**, porque o relógio interno conta segundos.

2) 1 hora = 60 min × 60 s = **3600 s**.

3) Bahia = UTC−3, então: −3 × 3600 = **−10 800 s**. É o valor de GMT_OFFSET.

4) O segundo argumento (0) é o horário de verão, que o Brasil não usa atualmente.

Parte 2 · A meteorologia por trás do painel

2.1 Temperatura e sensação térmica

A **temperatura do ar** é medida a 2 metros do chão e à sombra (`temperature_2m`). A **sensação térmica** (`apparent_temperature`) estima o que o corpo sente: umidade alta atrapalha o suor de evaporar (sentimos mais calor), vento ajuda a refrescar e o sol direto esquenta.

> **Analogia do dia a dia: a roupa molhada no varal**
>
> Num dia seco e com vento, a roupa seca rápido; num dia abafado, demora. O suor é a "roupa molhada" do corpo: quando não evapora, o calor não vai embora, e 31 °C parecem 34 °C.

2.2 Umidade relativa e ponto de orvalho

> **Analogia do dia a dia: a esponja e o copo gelado**
>
> O ar é como uma esponja que segura vapor d'água. Ar quente é uma esponja **grande**; ar frio, uma esponja **pequena**. **Umidade relativa** = quanto a esponja está cheia, em %. Se você esfria o ar, a esponja encolhe até não caber mais água e ela "pinga": é o copo gelado que fica suado. A temperatura em que isso acontece é o **ponto de orvalho**.

O ponto de orvalho é um termômetro de "abafamento" melhor que a umidade relativa, porque não depende da hora do dia. Regra prática: orvalho abaixo de 16 °C é confortável, entre 16 e 20 °C começa a pesar, acima de 21 °C é abafado. Como o DHT22 não mede o orvalho, o ESP32 **calcula** com a fórmula de Magnus:

$\gamma = \ln(UR/100) + (b \cdot T)/(c + T)$ e **$Td = (c \cdot \gamma)/(b - \gamma)$**, com b = 17,62 e c = 243,12 °C

Cálculo passo a passo: ponto de orvalho com T = 29,5 °C e UR = 72 %

Passo 1 – parte da umidade: ln(72/100) = ln(0,72) = **−0,3285**. Dividimos por 100 porque a fórmula usa a umidade como fração (0 a 1), não em %. O logaritmo aparece porque a pressão de vapor que o ar suporta cresce de forma exponencial com a temperatura; o ln "desfaz" a exponencial.

Passo 2 – parte da temperatura: b·T/(c+T) = 17,62 × 29,5 / (243,12 + 29,5) = 519,79 / 272,62 = **1,9067**. Esse termo mede o "tamanho da esponja" na temperatura atual.

Passo 3 – somar: γ = −0,3285 + 1,9067 = **1,5782**. Juntamos "o quanto está cheia" com "o tamanho da esponja".

Passo 4 – voltar para °C: Td = 243,12 × 1,5782 / (17,62 − 1,5782) = 383,69 / 16,0418 = **23,9 °C**. É a mesma fórmula do passo 2, só que resolvida "ao contrário" para achar a temperatura.

Conclusão: 23,9 °C de orvalho = tempo abafado, bem típico do Recôncavo/Agreste baiano. O número coincide com o que o próprio código calculou na simulação de teste (23,92).

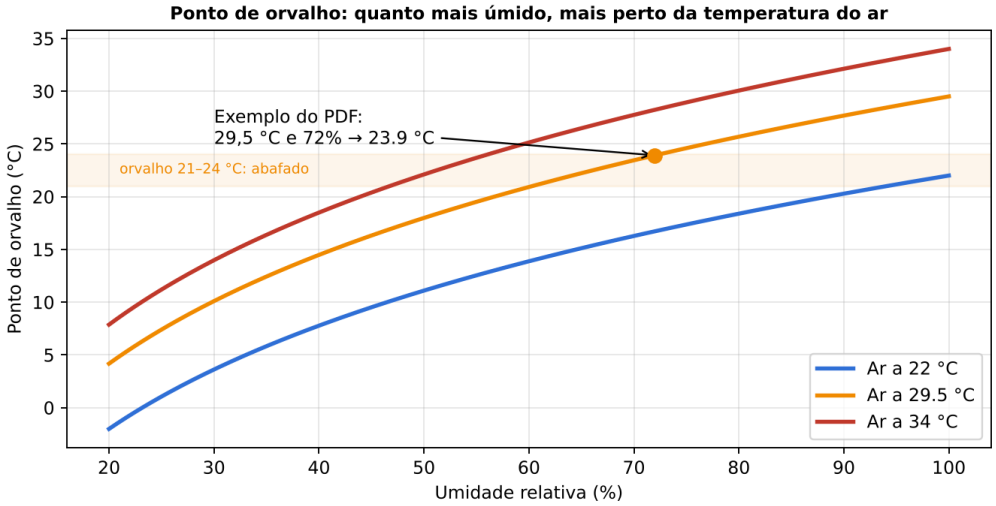


Figura 2 – Ponto de orvalho em função da umidade, para três temperaturas.

2.3 Pressão atmosférica

Analogia do dia a dia: a pilha de colchões

Imagine uma pilha enorme de colchões: o de baixo é o mais espremido, porque segura o peso de todos. O ar é igual: no nível do mar você está embaixo da "pilha" inteira de atmosfera. Subindo uma serra, sobra menos ar em cima de você, então a pressão cai.

O BMP180 mede a **pressão absoluta** no local (em pascal, Pa). Alagoinhas fica a cerca de 158 m de altitude, então a pressão lá é naturalmente menor que em Salvador. Para comparar com boletins (que usam o nível do mar), "corrigimos" a leitura para o nível do mar, usando a fórmula barométrica internacional:

$$p_0 = p / (1 - h/44330)^{5,255}$$

Cálculo passo a passo: pressão ao nível do mar a partir do BMP180

Passo 1 – Pa para hPa: o sensor deu 99 350 Pa. Dividimos por 100: **993,50 hPa**. Fazemos isso porque meteorologia usa hectopascal (1 hPa = 100 Pa), que também é igual ao antigo milibar.

Passo 2 – fração da altitude: h/44330 = 158/44330 = **0,003564**. O número 44 330 m é a "altura" de uma atmosfera padrão simplificada; a fração diz quanto dela está abaixo de nós.

Passo 3 – complemento: 1 − 0,003564 = **0,996436**.

Passo 4 – expoente: $0{,}996436^{5{,}255}$ = **0,98141**. O expoente 5,255 vem da física do ar (gravidade, massa molar do ar e queda de temperatura com a altitude). Na calculadora: use a tecla x^y.

Passo 5 – dividir: p_0 = 993,50 / 0,98141 = **1012,3 hPa**. Dividimos porque a pressão no alto é uma fração da do nível do mar; para voltar, desfazemos a multiplicação.

Conferência: a Open-Meteo informou 1012,4 hPa de pressão ao nível do mar. Diferença de 0,1 hPa: a correção funcionou.

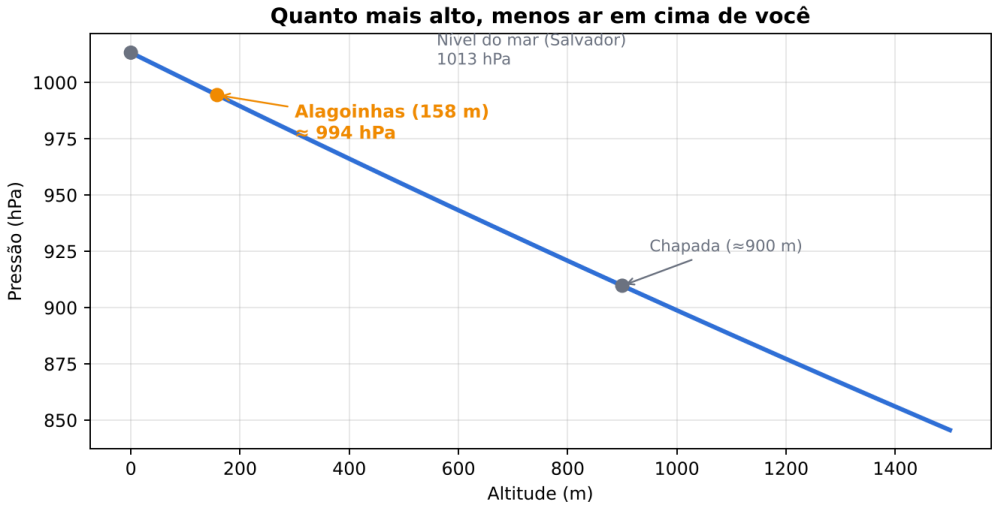


Figura 3 – Pressão diminui com a altitude. Alagoinhas fica perto de 994 hPa.

Para que serve? **Tendência**: pressão caindo vários hPa em poucas horas costuma indicar aproximação de chuva e instabilidade; subindo, tempo firme. Esse é um ótimo próximo passo: guardar o histórico e desenhar um gráfico de tendência.

2.4 Vento: de graus para pontos cardeais

A API dá a direção de onde o vento **vem**, em graus: 0° = norte, 90° = leste, 180° = sul, 270° = oeste. Dividimos a rosa dos ventos em 8 fatias de 45° cada: N, NE, L, SE, S, SO, O, NO.

Cálculo passo a passo: 110° vira qual direção?

Passo 1: somar 22,5°: 110 + 22,5 = 132,5. Por quê? Cada fatia é *centrada* no seu ponto cardeal: o "N" vai de −22,5° a +22,5°. Somar meia fatia faz cada fatia começar no múltiplo de 45, e aí basta dividir.

Passo 2: dividir pelo tamanho da fatia: 132,5 / 45 = 2,94. **Passo 3:** pegar a parte inteira: 2.

Passo 4: posição 2 da lista {N, NE, **L**, SE, ...} = **L** (leste). O `% 8` no código faz 360° voltar a ser "N" (8 % 8 = 0). Vento de leste/sudeste é o mais comum na Bahia (ventos alísios).

2.5 Índice UV

Analogia do dia a dia: a dose de remédio

O índice UV é a "dose" de radiação ultravioleta que chega até você. Dose baixa não faz mal; dose alta, por tempo suficiente, queima a pele. Ele vai de 0 a 11+ e, perto do equador, como em Alagoinhas, passa de 10 em dias de céu limpo.

UV	Categoria	Cor no painel	Recomendação mostrada
0 a 2	Baixo	verde	Sem cuidado especial
3 a 5	Moderado	amarelo	Protetor e boné
6 a 7	Alto	laranja	Protetor, evite 10h–16h
8 a 10	Muito alto	vermelho	Evite sol 10h–16h!
11 ou mais	Extremo	roxo	Evite sol 10h–16h!

2.6 Qualidade do ar (AQI)

O AQI (Índice de Qualidade do Ar) dos EUA junta vários poluentes num número de 0 a 500. Os principais aqui são as **partículas** PM2.5 (menores que 2,5 micrômetros, entram fundo no pulmão: fumaça, queimadas) e PM10 (poeira). Em Alagoinhas o ar costuma ser bom, mas o índice sobe em períodos de queimadas. Os dados vêm de um modelo do serviço europeu Copernicus (CAMS) com resolução de cerca de 45 km: é uma estimativa regional, não uma medição na sua rua.

AQI	Categoria	Cor
0–50	Bom	verde
51–100	Moderado	amarelo
101–150	Ruim para grupos sensíveis	laranja
151–200	Ruim	vermelho
201–300	Muito ruim	roxo
301+	Perigoso	vinho

2.7 Os códigos do tempo (WMO)

A API não manda "nublado" em texto: manda um **número** padronizado pela Organização Meteorológica Mundial. O código traduz o número em texto (`weatherText`) e em ícone (`weatherIcon`).

Código	Significado	Código	Significado
0	Céu limpo	61 / 63 / 65	Chuva fraca / moderada / forte
1 / 2 / 3	Poucas nuvens / parcialmente nublado / nublado	80 / 81 / 82	Pancadas fracas / moderadas / fortes
45 / 48	Neblina	95	Trovoada
51 / 53 / 55	Garoa	96 / 99	Trovoada com granizo

Parte 3 · A eletrônica

3.1 ESP32: o cérebro

O ESP32 é um microcontrolador de dois núcleos a 240 MHz, com Wi-Fi e Bluetooth embutidos, 520 KB de RAM e (na DevKit) 4 MB de memória flash. Comparado ao Arduino Uno (16 MHz, 2 KB de RAM, sem Wi-Fi), é muito mais forte, por isso consegue baixar e interpretar o JSON e desenhar numa tela colorida ao mesmo tempo.

Atenção: o ESP32 trabalha com 3,3 V

Os pinos do ESP32 **não toleram 5 V**. Por isso todos os módulos deste projeto são alimentados no pino **3V3**. No Arduino Uno você ligaria em 5 V; aqui, não.

Cálculo passo a passo: quanto do ESP32 o projeto usa

O compilador informou: programa = 974 237 bytes de 1 310 720 disponíveis. 974 237 / 1 310 720 = 0,743 → **74%** da flash.

Variáveis globais = 49 944 bytes de 327 680 → **15%** da RAM. Sobra bastante espaço para o JSON (cerca de 1,5 KB de texto) e para crescer o projeto.

3.2 Barramentos: SPI e I2C

Analogia do dia a dia: linha telefônica privativa x grupo de WhatsApp

SPI é como ter uma linha telefônica só para cada aparelho: muito rápida, mas gasta mais fios (cada dispositivo precisa do seu próprio fio "CS", que é como discar o número dele). Ideal para a tela colorida, que recebe muitos dados.

I2C é como um grupo de WhatsApp: só dois fios (SDA = dados, SCL = relógio) para todo mundo, e cada um responde pelo seu **nome** (endereço). A OLED atende pelo endereço **0x3C** e o BMP180 pelo **0x77**. Como os nomes são diferentes, os dois dividem os mesmos fios sem confusão.

	SPI (tela TFT)	I2C (OLED + BMP180)
Fios	MOSI, MISO, SCK + CS e D/C por aparelho	SDA e SCL compartilhados
Velocidade típica	dezenas de MHz	100 kHz a 400 kHz
Quem fala com quem	escolhido pelo pino CS	escolhido pelo endereço
Pinos no ESP32	MOSI 23, SCK 18, MISO 19, CS 15, D/C 2, RST 4	SDA 21, SCL 22

Cálculo passo a passo: por que a tela precisa de SPI

Uma tela cheia tem 320 × 240 = **76 800 pixels**. Cada pixel ocupa 2 bytes (16 bits): 76 800 × 2 = **153 600 bytes**.

Cada byte gasta cerca de 9 pulsos de relógio no I2C (8 bits + confirmação). A 400 kHz: 153 600 × 9 / 400 000 ≈ **3,5 s** por tela. Lento demais.

No SPI a 40 MHz (8 pulsos por byte): 153 600 × 8 / 40 000 000 ≈ **0,03 s**. Cerca de 100 vezes mais rápido.

3.3 Sensores

DHT22 (AM2302): temperatura e umidade

Dentro dele há um capacitor cujo valor muda com a umidade e um termistor. Um pequeno chip converte tudo e envia por **um único fio** de dados, num protocolo próprio baseado na duração dos pulsos. Faixa: −40 a 80 °C (±0,5 °C) e 0 a 100% (±2 a 5%). Limite importante: **no máximo uma leitura a cada 2 segundos**; por isso lemos a cada 5 s.

BMP180: pressão e temperatura

Tem uma membrana microscópica de silício que deforma com a pressão do ar, alterando resistências (efeito piezorresistivo). Comunica por I2C no endereço 0x77. Usamos a biblioteca do BMP085, seu antecessor, que é compatível. A temperatura dele serve para compensar a medição de pressão; ela costuma ficar um pouco diferente da do DHT22 por estar perto de outros componentes.

3.4 As telas

TFT ILI9341 2,8": cores em 16 bits (RGB565)

Cada pixel tem três "lâmpadas": vermelho, verde e azul. Em vez de 24 bits (8 por cor), a tela usa 16 bits para economizar: 5 para vermelho, 6 para verde e 5 para azul. O verde ganha um bit a mais porque nosso olho é mais sensível a ele.

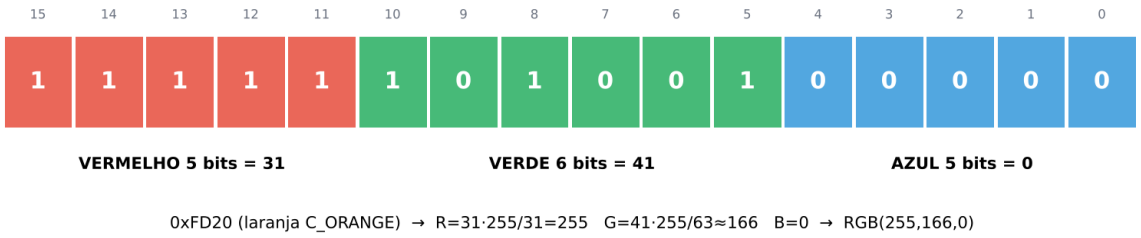


Figura 4 – Como a cor laranja 0xFD20 é guardada em 16 bits.

Cálculo passo a passo: converter a cor de fundo RGB(10, 14, 24) em RGB565

Passo 1 – reduzir cada cor: o vermelho tem 5 bits (0 a 31), então dividimos por 8 (que é 2^3, pois perdemos 3 bits): 10 / 8 = 1. O verde tem 6 bits (0 a 63): 14 / 4 = 3. O azul tem 5 bits: 24 / 8 = 3. (Divisões inteiras, descartando o resto.)

Passo 2 – posicionar: o vermelho fica nos 5 bits mais altos (desloca 11 posições), o verde no meio (desloca 5), o azul no fim. Em decimal: 1 × 2048 + 3 × 32 + 3 = 2048 + 96 + 3 = **2147**.

Passo 3 – hexadecimal: 2147 = 8 × 256 + 6 × 16 + 3 → **0x0863**, que é o `C_BG` do código. No código isso aparece como `(R >> 3) << 11 | (G >> 2) << 5 | (B >> 3)`.

OLED SH1107 128×128

Na OLED cada pixel é um pequeno LED orgânico que acende sozinho: não há luz de fundo, o preto é preto de verdade e o contraste é altíssimo. É monocromática (aceso ou apagado), então cada pixel cabe em 1 bit.

Cálculo passo a passo: a memória que a OLED ocupa no ESP32

Usamos o modo "buffer completo" (o _F_ no nome da classe): o ESP32 desenha a imagem inteira na RAM e manda de uma vez.

128 × 128 = 16 384 pixels. Como 1 byte = 8 bits = 8 pixels: 16 384 / 8 = **2048 bytes (2 KB)**. No ESP32 isso é 0,6% da RAM; no Arduino Uno seria a RAM inteira (por isso no Uno se usa o modo por páginas, mais lento).

3.5 Botões e resistor de pull-up

Analogia do dia a dia: a porta com mola

Um pino de entrada solto é como uma porta sem trinco balançando ao vento: às vezes lê 0, às vezes 1 (fica "flutuando"). O **resistor de pull-up** é uma mola que mantém a porta fechada em 1 (3,3 V). Apertar o botão é empurrar a porta: liga o pino ao GND e ele lê 0.

O ESP32 tem esse resistor dentro (cerca de 45 kΩ), ativado com `INPUT_PULLUP`. Por isso o botão vai só entre o pino e o GND, sem resistor externo, e "apertado" significa **LOW**. Os contatos metálicos "quicam" por alguns milissegundos ao fechar (*bounce*); o código ignora um novo aperto por 200 ms (*debounce*).

3.6 LED RGB e o cálculo dos resistores

Um LED RGB são três LEDs num só encapsulamento. No de **cátodo comum**, o terminal negativo (o mais comprido) é compartilhado e vai ao GND; cada cor acende com nível alto no seu pino. Amarelo = vermelho + verde acesos juntos.

Cálculo passo a passo: corrente em cada cor com resistor de 220 Ω (Lei de Ohm)

Ideia: o resistor fica com a tensão que "sobra" depois do LED. $I = (V_{pino} - V_{LED}) / R$.

Vermelho (queda típica 2,0 V): (3,3 − 2,0) / 220 = 1,3 / 220 = 0,0059 A = **5,9 mA**. Bom brilho, abaixo do limite seguro do pino do ESP32 (recomenda-se até cerca de 20 mA por pino).

Verde e azul (queda típica 3,0 V): (3,3 − 3,0) / 220 = 0,3 / 220 = **1,4 mA**. Funciona, mas mais fraco.

Ajuste no físico: para equilibrar, use 100 Ω no verde e no azul: 0,3 / 100 = 3 mA. Não use menos que isso sem medir, pois a queda real varia de LED para LED. (No Wokwi qualquer valor funciona.)

Cor do LED	Significado	Exemplo
Verde fixo	Tudo normal	nenhum alerta
Amarelo fixo	Atenção	sensação 33 °C, UV máximo alto, chuva provável
Vermelho piscando	Alerta	trovoada, chuva forte, sensação 38 °C, rajadas acima de 60 km/h
Azul piscando	Sem conexão e sem dados	Wi-Fi caiu antes da primeira leitura

Parte 4 · Montagem no Wokwi

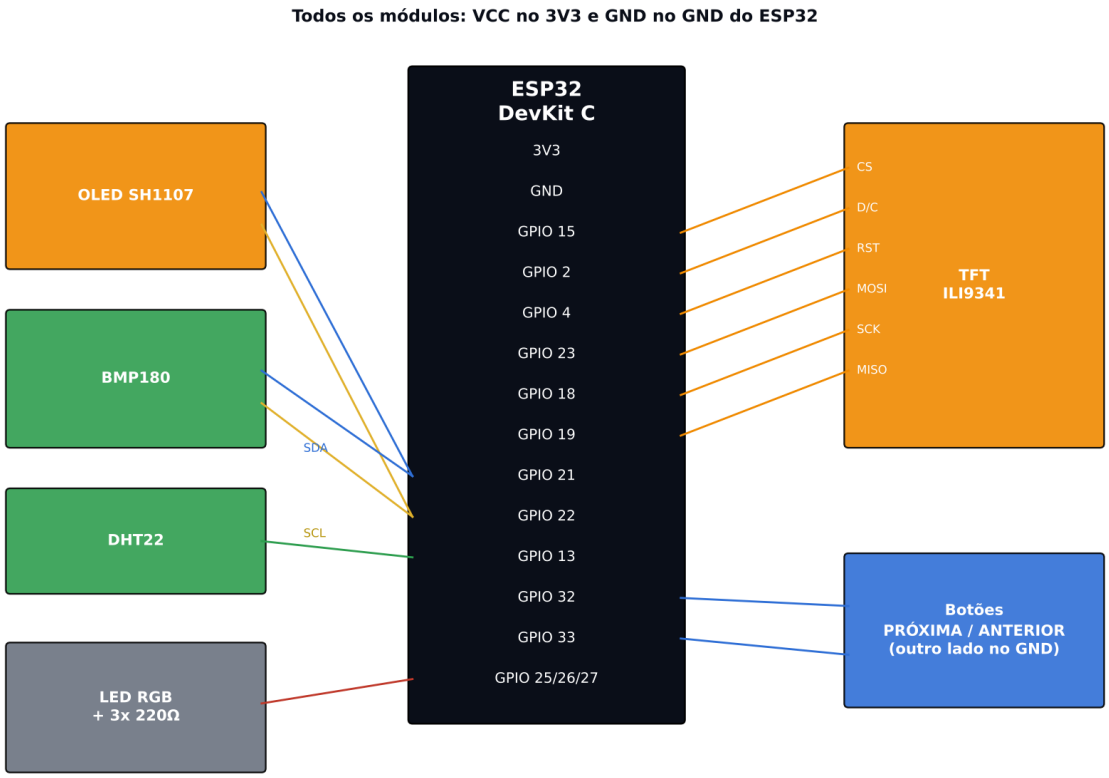


Figura 5 – Esquema de ligações.

Componente	Pino do componente	Pino do ESP32	Observação
TFT ILI9341	VCC / LED	3V3	LED é a luz de fundo
	GND	GND	
	CS	GPIO 15	escolhe a tela no SPI
	D/C	GPIO 2	comando ou dado
	RST	GPIO 4	reset (não simulado no Wokwi)
	MOSI / SCK / MISO	23 / 18 / 19	SPI padrão do ESP32 (VSPI)
OLED SH1107	VCC / GND	3V3 / GND	
	SDA / SCL	21 / 22	I2C, endereço 0x3C
BMP180	VCC / GND	3V3 / GND	
	SDA / SCL	21 / 22	mesmo I2C, endereço 0x77
DHT22	VCC / GND	3V3 / GND	no físico: resistor 10 kΩ do dado ao 3V3 (módulos já têm)
	SDA (dados)	GPIO 13	
Botão PRÓXIMA	um lado / outro lado	GPIO 32 / GND	pull-up interno
Botão ANTERIOR	um lado / outro lado	GPIO 33 / GND	pull-up interno
LED RGB	R / G / B	25 / 26 / 27	cada um via resistor 220 Ω

Componente	Pino do componente	Pino do ESP32	Observação
	COM (cátodo)	GND	

Passo a passo para rodar

1. Entre em **wokwi.com**, faça login (assim o projeto fica salvo) e clique em **New Project → ESP32** (modelo Arduino).
2. Abra a aba **sketch.ino**, apague tudo e cole o conteúdo do arquivo `sketch.ino` que você recebeu.
3. Abra a aba **diagram.json**, apague tudo e cole o arquivo `diagram.json`. As peças e fios aparecem sozinhos. Se algumas peças ficarem sobrepostas, arraste-as; os fios acompanham.
4. Clique na seta ao lado das abas → **New file** → nome `libraries.txt` e cole o arquivo de bibliotecas. (Ou adicione uma a uma pelo **Library Manager**.)
5. Clique no botão verde ▶. A primeira compilação demora um pouco. Acompanhe o **Serial Monitor**: vai aparecer "[WiFi] conectado!", depois "[CLIMA] ..." com a temperatura real de Alagoinhas.
6. Durante a simulação, **clique no DHT22 e no BMP180** para abrir os controles deslizantes e mudar temperatura, umidade e pressão. Coloque o DHT22 em 36 °C e veja o alerta "Casa muito quente" e o LED ficar amarelo.
7. Aperte os botões para navegar. Depois de 30 s sem apertar, a troca automática volta.

> **Salvar no Nexora**
>
> Clique em **Save** e renomeie o projeto para algo como "Nexora – Estação Meteorológica v1". Copie o link do projeto e cole na sua nota do Obsidian deste módulo.

Parte 5 · O código explicado

5.1 O mapa do programa

Bloco	O que faz
CONFIGURAÇÃO	rede Wi-Fi, coordenadas, altitude, fuso, intervalos de atualização (o que você mais vai mexer)
PINOS e OBJETOS	nomes para os pinos e criação dos "controladores" de cada peça (tft, oled, dht, bmp)
CORES	paleta em RGB565
DADOS GUARDADOS	structs Weather, Air, Local e a lista de alertas: a "memória" do que foi lido
UTILIDADES	hora, dia da semana, direção do vento, textos do tempo, orvalho, pressão ao nível do mar
INTERNET	httpGetJson, fetchWeather, fetchAir: buscam e interpretam os dados
SENSORES / ALERTAS / LED	leitura local, regras de alerta, cor do LED
DESENHO	ajudantes de texto e ícones, moldura, as 6 páginas, a OLED
BOTÕES / WI-FI / setup / loop	navegação, conexão e o "maestro" que coordena tudo

5.2 Struct: uma ficha com vários campos

Em vez de dezenas de variáveis soltas (temp, hum, wind...), agrupamos tudo numa **struct**, que é como uma pasta com várias folhas. `wx.temp` lê-se "a temperatura da pasta wx". Fica organizado e fácil de passar adiante.

```
struct Local {
  bool  dhtOk = false, bmpOk = false;   // o sensor respondeu?
  float t = NAN, h = NAN;               // DHT22
  float tb = NAN, p = NAN, p0 = NAN;    // BMP180
  float dew = NAN;                      // ponto de orvalho calculado
} loc;
```

NAN ("not a number") marca "ainda não tenho esse dado". As telas testam `isnan()` e mostram "--" em vez de um número falso.

5.3 O loop que nunca trava: millis()

> **Analogia do dia a dia: o cozinheiro com vários timers**
>
> Um cozinheiro iniciante põe o arroz no fogo e fica parado olhando 20 minutos (isso é o `delay()`): enquanto isso, o feijão queima. O cozinheiro esperto anota a hora em que pôs cada panela e, a cada volta pela cozinha, olha o relógio: "já deu 20 minutos do arroz? não... já deu 5 do feijão? sim, mexe". Ninguém fica parado.

`millis()` devolve quantos milissegundos se passaram desde que o ESP32 ligou. Cada tarefa guarda quando rodou pela última vez e compara:

```
if (now - lastSensor >= SENSOR_EVERY) {   // já passaram 5000 ms?
  lastSensor = now;                       // anota a "hora" desta vez
  readSensors();                          // faz a tarefa
}
```

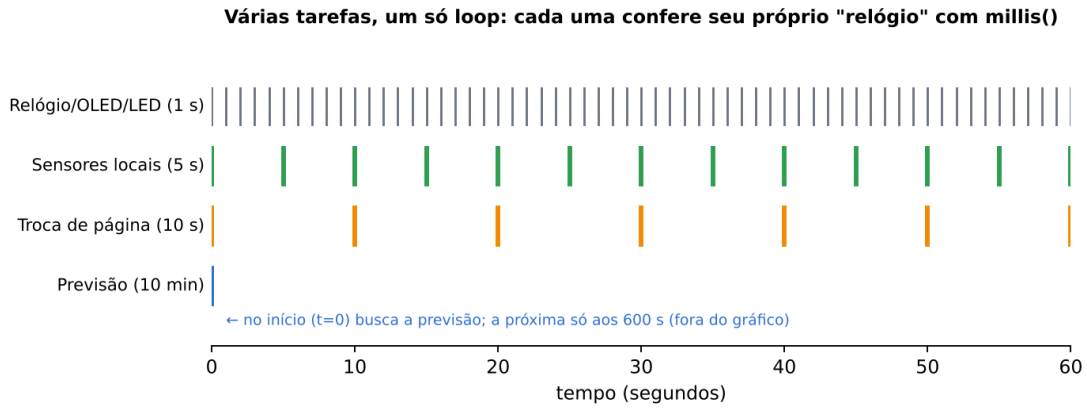


Figura 6 – Cada tarefa no seu ritmo, dentro do mesmo loop().

Cálculo passo a passo: os intervalos em milissegundos

Previsão: 10 min × 60 s × 1000 ms = **600 000 ms**. O `UL` em `10UL * 60UL * 1000UL` força a conta em *unsigned long* (32 bits sem sinal), para não estourar em placas onde *int* tem 16 bits.

Qualidade do ar: 30 × 60 × 1000 = **1 800 000 ms**. O dado do ar só é atualizado de hora em hora pelo serviço, então pedir a cada 30 min é suficiente e educado com a API gratuita.

Usar `now - last >= intervalo` (e não `now >= last + intervalo`) continua certo mesmo quando millis() "dá a volta" no zero depois de cerca de 49,7 dias (2^{32} ms), por causa da aritmética sem sinal.

Exceção consciente: o download HTTPS trava o loop por 1 a 3 segundos a cada 10 minutos. Para este projeto é aceitável; numa versão futura dá para mover o download para o segundo núcleo do ESP32 com FreeRTOS.

5.4 Regras de alerta

A função `evaluateAlerts()` refaz a lista do zero sempre que chegam dados novos. Cada regra tem um nível (1 = atenção, 2 = alerta) e o nível geral é o maior encontrado. Mudar os limites é só trocar os números:

Situação	Atenção (1)	Alerta (2)
Sensação térmica	≥ 33 °C	≥ 38 °C
UV	máximo do dia ≥ 8	agora ≥ 8
Rajadas de vento	≥ 40 km/h	≥ 60 km/h
Chance de chuva nas próximas 6 h	≥ 70%	–
Chuva prevista no dia	–	≥ 30 mm
Tempo agora	–	trovoada (95+), chuva forte (65, 82)

Situação	Atenção (1)	Alerta (2)
Qualidade do ar (AQI)	> 100	> 150
Temperatura dentro de casa (DHT22)	≥ 35 °C	–
Wi-Fi	desconectado	–

```
if (wx.feels >= 38)      addAlert(2, "Calor extremo: sensacao %.0f C", wx.feels);
else if (wx.feels >= 33) addAlert(1, "Calor forte: sensacao %.0f C", wx.feels);
```

O `%.0f` é um "espaço reservado" do `snprintf`: troca pelo número com 0 casas decimais. Os textos não têm acento porque a fonte embutida das telas só tem os caracteres básicos.

5.5 Desenhando o gráfico das 12 horas

O gráfico precisa transformar **graus Celsius** em **pixels**. É uma regra de três: a temperatura mínima vai para o fundo do gráfico (y = 172) e a máxima para o topo (y = 48). Lembre que na tela o **y cresce para baixo**.

Cálculo passo a passo: em que altura fica o ponto de 29,1 °C (18h)?

Passo 1 – margens: menor temperatura das 12 h = 24,2 °C; maior = 31,2 °C. O código dá 1 grau de folga e arredonda: tMin = floor(23,2) = **23**; tMax = ceil(32,2) = **33**. A folga evita que pontos encostem nas bordas.

Passo 2 – fração: (29,1 − 23) / (33 − 23) = 6,1 / 10 = **0,61**. Ou seja, 61% do caminho entre o mínimo e o máximo.

Passo 3 – para pixels: altura útil = 172 − 48 = 124 px. 0,61 × 124 = 75,6 → **75** (o (int) corta a parte decimal).

Passo 4 – inverter o eixo: como y cresce para baixo, subtraímos do fundo: 172 − 75 = **y = 97**.

As barras azuis de chance de chuva usam a mesma ideia: altura = 124 × probabilidade / 100. 55% → 68 px.

5.6 O arco do sol: seno e cosseno

Na página "Sol e UV" o solzinho anda num semicírculo do nascer (esquerda) ao pôr (direita). Para achar a posição, transformamos o horário num ângulo e o ângulo em coordenadas com trigonometria.

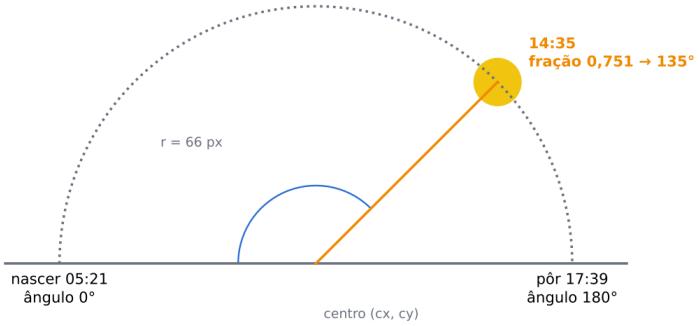


Figura 7 – Posição do sol às 14:35.

Cálculo passo a passo: onde desenhar o sol às 14:35

Passo 1 – tudo em minutos: nascer 05:21 = 5×60+21 = **321**; pôr 17:39 = 17×60+39 = **1059**; agora 14:35 = **875**. Minutos permitem fazer contas com um número só.

Passo 2 – fração do dia claro: (875 − 321) / (1059 − 321) = 554 / 738 = **0,751** (75% do dia já passou).

Passo 3 – ângulo: 0,751 × π rad = 2,359 rad = **135°**. Meia volta (180°, ou π radianos) representa o dia inteiro.

Passo 4 – x: x = cx − cos(135°) × r = 82 − (−0,707) × 66 = 82 + 46,7 = **129**. O sinal de menos faz o ângulo 0° ficar à *esquerda*, onde está o nascer.

Passo 5 – y: y = cy − sin(135°) × r = 150 − 0,707 × 66 = 150 − 46,7 = **103**. Subtrai porque o y da tela cresce para baixo.

Bônus: duração do dia = 738 min = 12 h e 18 min (738 / 60 = 12, resto 18), o "Dia: 12h18min" da tela.

5.7 Dia da semana sem calendário

A API manda datas como "2026-10-12", mas o painel mostra "Seg". A função `weekday0f` usa o **algoritmo de Sakamoto**, que calcula o dia da semana com somas e divisões inteiras. A tabela `{0,3,2,5,0,3,5,1,4,6,2,4}` guarda o "deslocamento" de cada mês; janeiro e fevereiro contam como meses do ano anterior para o 29 de fevereiro cair no final. Teste: 12/10/2026 → resultado 1 = segunda-feira.

Parte 6 · As telas do painel

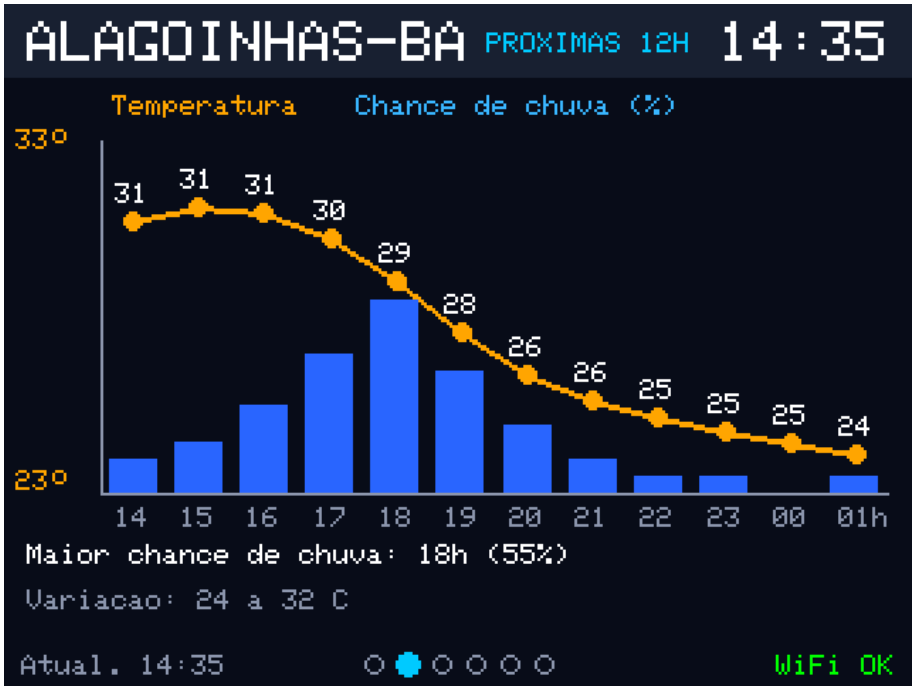
As imagens abaixo foram geradas pelo próprio código do projeto (as mesmas funções de desenho, rodando num computador) alimentado com **dados de exemplo** no formato da Open-Meteo. No Wokwi você verá os números reais do momento.

Página 1 – AGORA



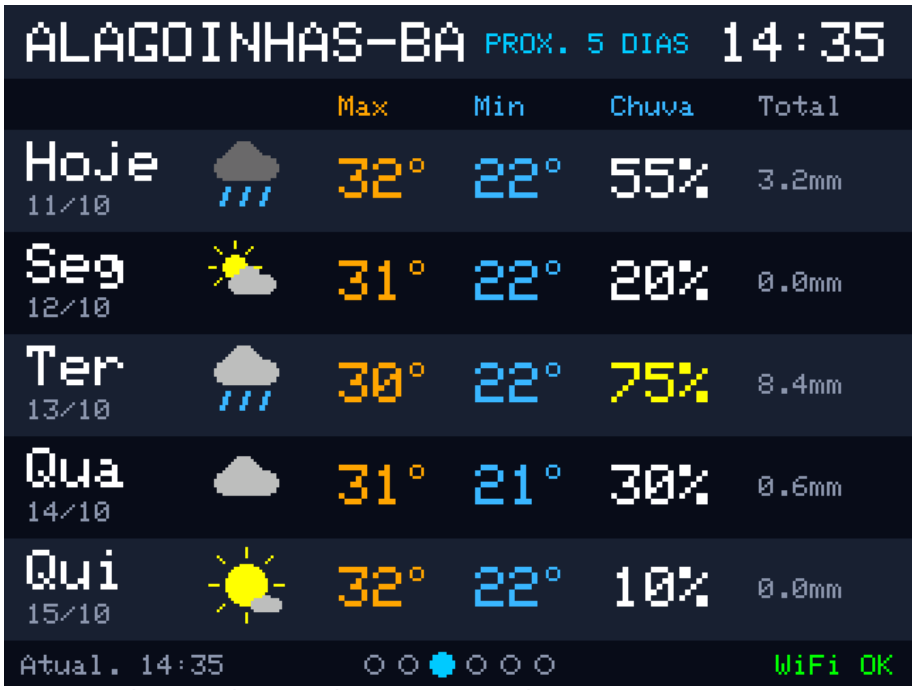
Ícone e texto do tempo, temperatura grande, máxima e mínima do dia e seis cartões: sensação térmica (laranja quando ≥ 33 °C), umidade, vento com direção, rajadas, pressão ao nível do mar e nebulosidade.

Página 2 – PRÓXIMAS 12H



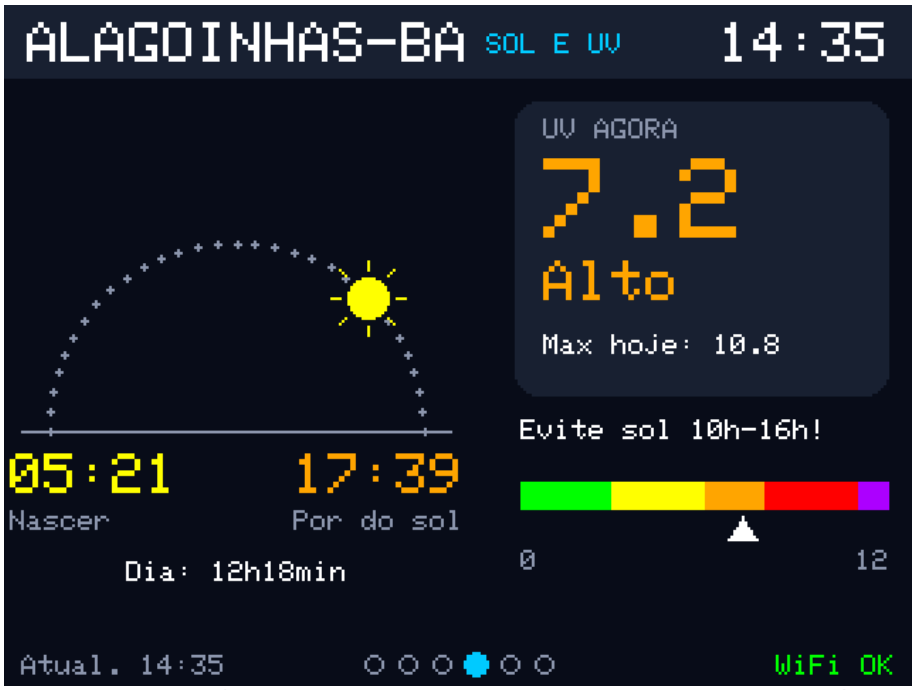
Linha laranja = temperatura de cada hora; barras azuis = chance de chuva. Embaixo, a hora mais provável de chover e a variação de temperatura.

Página 3 – PRÓXIMOS 5 DIAS



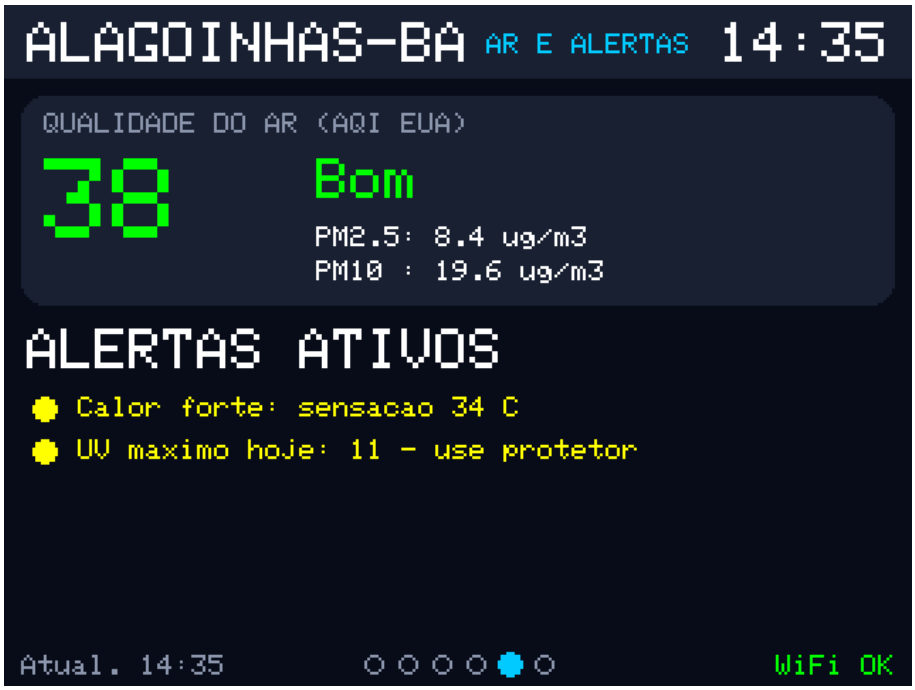
Dia da semana, data, ícone, máxima, mínima, chance máxima de chuva (amarela quando ≥ 60%) e o total de chuva previsto em mm.

Página 4 – SOL E UV



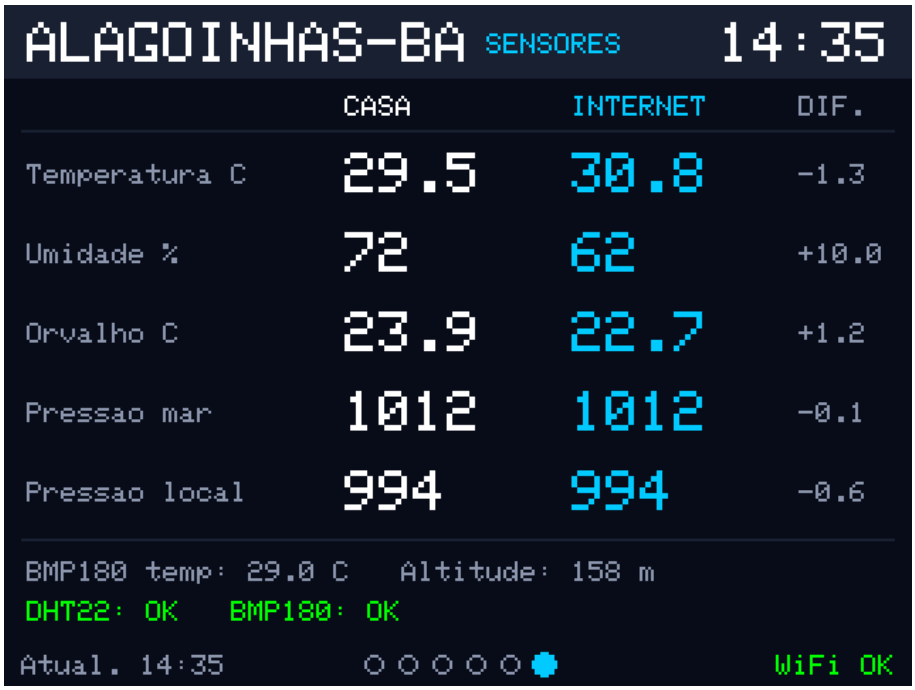
Arco do sol com nascer, pôr e duração do dia; UV atual com cor da categoria, máximo do dia, recomendação e escala de 0 a 12.

Página 5 – AR E ALERTAS



AQI com categoria e cor, PM2.5 e PM10; abaixo, a lista de alertas ativos (amarelo = atenção, vermelho = alerta).

Página 6 – SENSORES



Casa (DHT22/BMP180) x internet, com a diferença. Mostra o ponto de orvalho e a pressão corrigida calculados pelo ESP32, e o estado de cada sensor.

OLED – resumo permanente



Relógio, temperatura e ícone da internet, condição do tempo, temperatura e umidade da casa e, na faixa acesa de baixo, os alertas se revezando a cada 3 segundos.

Como ler a diferença Casa x Internet

É normal a casa ter temperatura e umidade diferentes: a internet informa o ar **externo** estimado por modelo numa grade de alguns quilômetros. Já a **pressão** deve ficar muito parecida (ar se equilibra rápido); diferença grande nela indica altitude configurada errada ou sensor com problema.

Parte 7 · Do Wokwi para o mundo real

7.1 Lista de compras

Preços em reais, aproximados. Apenas o ESP32 foi conferido numa loja online em outubro de 2026; os demais são **faixas estimadas** do mercado brasileiro. Confira antes de comprar: varia muito entre lojas, Mercado Livre e importação.

#	Item	Qtd	Preço aprox. (R$)	Observação
1	ESP32 DevKitC V4 WROOM-32 (38 pinos, CP2102)	1	57 – 60	conferido (Usinainfo, out/2026)
2	Display TFT 2,8" SPI 240×320 ILI9341	1	60 – 110 (est.)	com ou sem touch, ambos servem
3	Display OLED 128×128 I2C, SH1107 (1,12" ou 1,5")	1	50 – 130 (est.)	mais difícil de achar; muitas vezes importado
4	Sensor DHT22 / AM2302 (de preferência em módulo)	1	30 – 50 (est.)	módulo já tem o resistor de 10 kΩ
5	Sensor BMP180 (GY-68)	1	12 – 25 (est.)	ou BMP280, ver 7.2
6	LED RGB 5 mm cátodo comum	1	1 – 3 (est.)	confira se é cátodo comum
7	Resistores 220 Ω (e 2× 100 Ω opcionais)	3	1 – 3 (est.)	ver cálculo da seção 3.6
8	Botão táctil 12 mm	2	2 – 5 (est.)	
9	Protoboard 830 pontos	1	15 – 25 (est.)	o ESP32 de 38 pinos ocupa quase toda
10	Kit de jumpers (macho-macho e macho-fêmea)	1	15 – 25 (est.)	
11	Cabo micro-USB **de dados**	1	10 – 20 (est.)	cabo só de carga não grava
	Total estimado		**≈ 250 – 450**	

> **Economizando**
>
> Se o orçamento apertar, comece só com ESP32 + OLED + DHT22: já dá um mostrador com dados reais. Uma OLED **SH1106 1,3" 128×64** é bem mais comum e barata que a SH1107; o código precisa trocar o construtor para `U8G2_SH1106_128X64_NONAME_F_HW_I2C` e reorganizar a OLED para metade da altura.

7.2 O que muda na montagem física

Item	No Wokwi	No físico
Wi-Fi	"Wokwi-GUEST", sem senha, canal 6	o nome e a senha da sua rede, canal **0** (automático)
OLED	controlador SH1107 genérico	se a imagem sair deslocada (comum na Grove da Seeed), mude `USE_SEEED_OLED` para 1

Item	No Wokwi	No físico
Pressão	BMP180	BMP280 é mais comum: biblioteca "Adafruit BMP280", `Adafruit_BMP280 bmp;`, `bmp.begin(0x76);` `readPressure()` continua em Pa
DHT22	simulado, valor ajustável	deixe longe do ESP32 e da tela (eles esquentam e enganam a leitura)
LED RGB	220 Ω em todas as cores	100 Ω no verde e no azul para brilho parecido
Alimentação	ilimitada	carregador USB de 5 V / 1 A ou mais; a TFT acesa consome bastante

Cuidados

Nunca ligue módulos em 5 V nos pinos de sinal do ESP32. Monte com o cabo USB **desconectado** e confira VCC/GND duas vezes antes de ligar: inverter alimentação queima o módulo. GPIO 2 também acende o LED azul da placa; isso é normal.

Parte 8 · Solução de problemas e próximos passos

Sintoma	Causa provável	O que fazer
Serial mostra "[HTTP] erro -1" ou -11	sem internet ou servidor demorou	espere: o código tenta de novo a cada 20 s
"[HTTP] erro 400"	nome de variável errado na URL	compare a URL com a documentação da Open-Meteo
"[JSON] erro: NoMemory"	resposta grande demais	diminua forecast_days ou forecast_hours
Relógio "--:--"	NTP ainda não respondeu	aguarde alguns segundos; o horário da API é usado no rodapé
DHT22: ERRO	fio de dados errado ou leituras rápidas demais	confira o GPIO 13; não leia com intervalo menor que 2 s
BMP180: ERRO	I2C mal ligado	confira SDA 21 / SCL 22; rode um "I2C scanner" no físico
TFT branca	CS/DC trocados ou sem alimentação	confira a tabela de ligações
OLED deslocada (físico)	variante do controlador	troque USE_SEEED_OLED
Botão troca 2 páginas	bounce	aumente 200 para 300 ms em pressed()

Próximos passos para o Nexora

- **Histórico e tendência da pressão:** guardar uma leitura por hora num vetor e desenhar um gráfico de 24 h (previsão "caseira" de chuva).
- **Automação de verdade:** usar os dados para agir. Ex.: relé liga o ventilador quando a casa passa de 30 °C; aviso para recolher a roupa quando a chance de chuva passar de 70%.
- **Página web:** o ESP32 pode servir uma página no Wi-Fi da casa, para ver o painel no celular.

- **MQTT / Home Assistant:** publicar as leituras para outros módulos do Nexora usarem (a cafeteira, as luzes...).
- **Cartão SD:** registrar os dados para estudar o clima de Alagoinhas ao longo do ano.
- **Segurança:** trocar setInsecure() pelo certificado raiz da Open-Meteo.

Glossário rápido

Termo	Significado
API	serviço que entrega dados para outros programas, seguindo regras documentadas
HTTP / HTTPS	protocolo de pedidos e respostas na web; o S significa criptografado (TLS)
JSON	formato de texto com campos "nome": valor, listas [ ] e objetos { }
NTP	protocolo para acertar o relógio pela internet
SPI / I2C	barramentos de comunicação entre chips (rápido com mais fios / lento com 2 fios e endereços)
RGB565	cor em 16 bits: 5 vermelho, 6 verde, 5 azul
Pull-up	resistor que mantém um pino em nível alto quando nada está ligado
Debounce	ignorar os "quiques" mecânicos de um botão
hPa	hectopascal = 100 Pa; unidade usual de pressão em meteorologia
Ponto de orvalho	temperatura em que o ar saturaria e a água condensaria
AQI	índice de qualidade do ar, de 0 (ótimo) a 500 (perigoso)
WMO	Organização Meteorológica Mundial; define os códigos do tempo

Fontes consultadas

Documentação da Open-Meteo (Weather Forecast API e Air Quality API: parâmetros, códigos WMO e faixas do AQI) – open-meteo.com/en/docs; Geocodificação Open-Meteo para Alagoinhas (−12,13556; −38,41917; 158 m); documentação do Wokwi (peças suportadas, ILI9341, SH1107, BMP180, LED RGB) – docs.wokwi.com; código do chip SH1107 do Wokwi e da biblioteca U8g2; preço do ESP32 – usinainfo.com.br.